Pointer Arithmetic

Pointer Expression and Pointer Arithmetic

Similar to other variables , pointer variables can also be used in expressions.

For example, If ptr1 and ptr2 are pointers , then the following statements are valid.

```
#include<iostream>
using namespace std;
int main()
{
     int num1 = 2, num2 = 3, sum = 1, mul = 0, div = 1;
     int *ptr1, *ptr2;
     ptr1 = &num1;
     ptr2 = &num2;
     sum = *ptr1 + *ptr2;
     mul = sum * *ptr1;
     *ptr2 += 1;
     div = 9 + *ptr1 / *ptr2 - 30;
     cout << *ptr1 << endl;
     cout << *ptr2 << endl;
     cout << sum << endl;
     cout << mul << endl;
     cout << *ptr2 << endl;
     cout << div << endl;
     system("pause");
     return 0;
}
```

Rules

1. Integers may be added or subtracted from a pointer.
2. We can also subtract one pointer from another pointer.
3. We can also use shorthand operator with pointer variable as we use with other variables.
4. C++ also allows us to compare pointers by relational operators in the expressions.
5. When using pointers unary increment(++) and decrement (--) operators have greater precedence than the dereference operator(*).
6. Both these operators have a special behavior when used as suffix.
7. In that case the expression is evaluated with the value it had before being increased.
8. The expression *ptr++ is equivalent to *(ptr++) .
9. To increment the value of a variable whose address is stored in ptr , you should write (*ptr)++.

```
#include<iostream>
using namespace std;
int main()
{
    int num1 = 2, num2 = 3;
    int *p = &num1, *q = &num2;
    cout <<*p << endl;
    cout <<*q << endl;
    *p++;
    *q++;
    cout << *p << endl;
    cout << *q << endl;
    system("pause");
    return 0;
}
```

What will *p++ and *q++ do?
Because ++ has a higher precedence than * , both p and q are increased, but because increment operators(++) are used as postfix and not prefix, the value assigned to *p and *q before both p and q are increased.

NULL Pointers

1. NULL pointer is a special pointer value that does not point anywhere.
2. This means that a NULL pointer does not point to any valid memory address.
3. To declare NULL pointer you may use predefined constant NULL.

```
int *p = NULL;
```

4. You may also initialize a pointer as NULL pointer by using a constant 0, as follows:

```
int  *ptr; ptr =0;
```

Uses of NULL Pointer

1. A function that returns a pointer value can return a NULL pointer when it is unable to perform its task.

2. NULL pointers are used in situations where one of the pointers in the program points to different locations at different times.

3. In this situations it is always better to set it to a NULL pointer when it doesn't point anywhere valid, and to test to check if it's a NULL pointer before using it.

Note: A run-time error is generated if you try to dereference a NULL pointer.

Generic Pointers

1. A generic pointer is a pointer variable that has void as its data type. The void pointer or generic pointer ,is a special type of pointer that can be used to point to variables of any data types. It is declared like a normal pointer variable but using the void keyword as the pointer's data type.

For example:

void *ptr;

2. In C++ since we can not have a variable of void type, the void pointer will therefore not point to any data and thus cannot be dereferenced.

3. You need to typecast a void pointer (generic pointer) to another kind of pointer before using it.

4. Generic pointers are often used when you want a pointer to point different types at different times.

```
#include<iostream>
using namespace std;
int main()
{
    int x = 10;
    char ch = 'A';
    void *gp;
    gp = &x;
    cout << x << endl;
    cout << *gp<<endl;
    gp = &ch;
    cout << ch << endl;
    cout << *gp << endl;
    system("pause");
    return 0;
}
```

This is an error we cannot dereference a void pointer. If we want to dereference a void pointer we need to type cast a generic pointer to another kind of pointer before using it.

How to dereference a generic pointer

```
#include<iostream>
using namespace std;
int main()
{
    int x = 10;
    char ch = 'A';
    void *gp;
    gp = &x;
    cout << x << endl;
    cout << *(int*)gp<<endl;
    gp = &ch;
    cout << ch << endl;
    cout << *(char*)gp << endl;
    system("pause");
    return 0;
}
```

type casting of a generic pointer to another kind of pointer before using it.

Passing Arguments to Functions using Pointers

1. Pointers provide a mechanism to modify data declared in one function using code written in another function.
2. The calling function sends the addresses of variables and the called function must declare those incoming arguments as pointers.
3. In order to modify the variables sent by the caller, the called function must dereference the pointers that were passed to it.
4. Thus, passing pointers to a function avoids the overhead of copying data from one function to another.

To use pointers for passing arguments to a function, the programmer must do the following:

1. Declare the functions parameters as pointers.
2. Use the dereferenced pointers in the function body.
3. Pass the addresses as the actual argument when the function is called.

```cpp
#include<iostream>
using namespace std;
void  Add(int *a, int *b,int *t)
{
    *t = *a + *b;
}
int main()
{
    int num1, num2, total;
    cout << "enter the first Num:" << endl;
    cin >> num1;
    cout << "Enter the second num:" << endl;
    cin >> num2;
    Add(&num1, &num2, &total);
    cout << "Total = " << total<<endl;
    system("pause");
    return 0;
}
```

Pointers and Arrays

1. Array notation is a form of pointer notation.
2. The name of the array is the starting address of the array in memory. It is also known as the base address.

Now let us use a pointer variable given in the following statement:

```
int *ptr;
ptr=&arr[0];
```

Here ptr is made to point to the first element of the array.

```cpp
#include<iostream>
using namespace std;

int main()
{
    int arr[] = { 2, 55, 72, 18, 15 };
    int *ptr;
    cout << arr << endl;
    cout << &arr[0] << endl;
    cout << &arr << endl;
    ptr = arr;
    cout << ptr <<endl;
    system("pause");
    return 0;
}
```

If pointer ptr holds the address of first element in the array, then the address of successive elements can be calculated by writing ptr++.

```
#include<iostream>
using namespace std;
int main()
{
    int arr[] = { 2, 55, 72, 18, 15 };
    int *ptr;
    ptr = arr;
    cout << ptr <<endl;
    cout <<++ptr << endl;
    cout << &arr[1] << endl;
    system("pause");
    return 0;
}
```

Difference between array name and pointer

1. C++ does not allow array name to be used as an l value. Hence, array names cannot appear on the left side of assignment operator.

2. However you may declare a pointer variable of appropriate type that points to the first element of the array and use it as its lvalue.

The first code gives an error as the array name is being used as an lvalue for the ++ operator.
The second code shows the correct way of doing the same thing.

```
#include<iostream>
using namespace std;
int main()
{   int a[5], i;
    for (i = 0; i < 5; i++)
    {   *a = 0;
        a++;//Error
    }
    for (i = 0; i < 5; i++)
    {       cout << *(a + i) << endl;
    }
    system("pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{   int a[5], i,*pa;
pa = a;
    for (i = 0; i < 5; i++)
    {   *pa = i;
        pa++;
    }
    for (i = 0; i < 5; i++)
    {       cout << *(a + i) << endl;
    }
    system("pause");
    return 0;
}
```

An array cannot be assigned to another array. However, one pointer variable can be assigned to another pointer variable of the same type.

```
#include<iostream>
using namespace std;
int main()
{
    int a1[] = { 1, 2, 3, 4, 5 };
    int a2[5];
    a2 = a1;//Error
    system("pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{
    int a1[] = { 1, 2, 3, 4, 5 },a2[5];
    int *ptr1, *ptr2;
    ptr1 = a1;
    [redacted]
    ptr2 = ptr1;
    system("pause");
    return 0;
}
```

Array of Pointers

An array of pointers can be declared as:

```
int   *tr[10];
```

The above statement declares an array of 10 pointers where each of the pointer points to an integer variable.

```
#include<iostream>
using namespace std;
int main()
{
    int a1[] = { 1, 2, 3, 4, 5 };
    int a2[] = {2, 5, 6, 7, 8};
    int a3[] = { 6, 8, 12, 11, 3 };
    int *ptr[3] = { a1, a2, a3 };
    int i;
    for (i = 0; i < 3; i++)
    {
        cout << *ptr[i] << endl;
    }
    system("pause");
    return 0;
}
```